基于梯度下降的线性回归拟合

从零实现（NumPy）实验报告

项目	内容
数据集	data/train.csv（100 条，训练） data/test.csv（30 条，测试）
任务	用学习时长（hours）预测考试成绩（score）
模型	score = 6.2363 × hours + 33.0468
优化算法	小批量梯度下降（Mini-batch GD）+ 动量 + 学习率衰减
测试集表现	R^2 = 0.9314，RMSE = 4.5360 分，MAE = 3.8964 分
实现约束	不使用 scikit-learn / pandas，模型、损失、梯度、优化器全部手写

本报告中的所有数字与图片，均由 code/train.py 一次运行自动产生，并保存在 results/ 目录中（图片为 PNG，指标为 JSON/CSV），可随时复现核对。

一、方法介绍

1.1 模型形式

本任务只有 1 个输入特征（学习时长 hours），因此线性回归模型就是一条直线。为了后面扩展到多特征，代码里统一使用矩阵形式：

$$\hat{y} = X \cdot w + b$$

其中 $X \in R^{n\times 1}$ 是特征矩阵，$w \in R^{1\times 1}$ 是权重（斜率），$b \in R^{1\times 1}$ 是偏置（截距），$\hat{y}$ 是预测值。展开成标量写法即 $\hat{y}_i = w \cdot x_i + b$。

参数初始化：w 和 b 都初始化为 0。这是可行的，因为线性回归的均方误差损失函数是凸函数，不存在“全 0 初始化导致对称性无法打破”的问题（那是神经网络才需要担心的）。

1.2 损失函数的选择

主损失函数选用均方误差 MSE（Mean Squared Error）：

$$L(w, b) = (1/n)\ \Sigma_{i=1..n}\ (\hat{y}_i - y_i)^2$$

选择 MSE 的三个理由：

① 它是凸函数，梯度下降一定能到达全局最优，不存在局部最优陷阱；

② 它对参数可导，且导数形式简单（见下节），便于手写实现；

③ 它在统计上对应“高斯噪声假设下的极大似然估计”，有理论依据。

此外，为了对比研究，代码中还实现了 MAE（平均绝对误差）与 Huber 损失，共三种，用于第 2.6 节的“异常值鲁棒性”实验。

1.3 梯度推导（手写，不使用自动微分）

对单个样本，损失为 $L_i = (\hat{y}_i - y_i)^2$，而 $\hat{y}_i = w\,x_i + b$，用链式法则求偏导：

$$\partial L_i/\partial w = 2(\hat{y}_i - y_i) \cdot x_i\ ,\quad \partial L_i/\partial b = 2(\hat{y}_i - y_i)$$

对全部 n 个样本取平均，写成矩阵形式：

$$\partial L/\partial w = (2/n) \cdot X□(\hat{y} - y)\ ,\quad \partial L/\partial b = (2/n) \cdot \Sigma(\hat{y}_i - y_i)$$

梯度正确性验证（重要）：为避免“梯度写错但程序照样跑”的隐蔽错误，本实现写了两道校验：

① 闭式解对拍：单特征线性回归有精确解 $w^* = \Sigma(x_i - x□)(y_i - □) / \Sigma(x_i - x□)^2$，$b^* = □ - w^* x□$。在构造数据 $y = 3x + 1$ 上，闭式解精确返回 $w = 3.000000$、$b = 1.000000$。

② 数值梯度检查：用有限差分 $\partial L/\partial w \approx [L(w+\varepsilon) - L(w-\varepsilon)] / (2\varepsilon)$（$\varepsilon = 1e{-}6$）近似梯度，与解析梯度对比，实测相对误差约 1e−9 量级。

1.4 梯度下降算法及更新公式

代码用同一个函数实现了三种梯度下降，仅通过“批量大小 batch_size”一个参数切换：

变体	每步样本数	每轮更新次数	特点
批量梯度下降 BGD	全部 100 条	1 次/轮	梯度最准、曲线最平滑，但每轮只更新 1 次，最慢

小批量梯度下降 MBGD	16 条	7 次/轮	兼顾梯度平稳与更新频率，深度学习中的默认选择
随机梯度下降 SGD	1 条	100 次/轮	每步极便宜、前期下降极快，但梯度抖动剧烈

基础更新公式（以 batch 为单位）：

$$w \leftarrow w - \eta \cdot \partial L_{batch}/\partial w\ ,\quad b \leftarrow b - \eta \cdot \partial L_{batch}/\partial b$$

注意梯度是对当前批量求平均（除以批量大小），这样三种批量模式下的学习率尺度才一致。

在基础公式之上，还实现了两个加速技巧：

① 动量 Momentum：把更新量看作“小球滚下山”的速度，带惯性：

$$v \leftarrow \beta \cdot v + (1-\beta) \cdot g\ ,\quad w \leftarrow w - \eta \cdot v\quad (\beta = 0.9)$$

其有效步长约为 $\eta/(1-\beta)$。这个结论很重要——它意味着加动量相当于把学习率放大 10 倍，因此动量必须与小一些的学习率搭配使用（实验 F 会定量验证这一点）。

② 学习率衰减 LR Decay：每个 epoch 让学习率乘上一个衰减系数：

$$\eta_t = \eta_0 \cdot decay^{t-1}\quad (decay = 0.998)$$

前期步子大、下降快，后期步子小、能收敛得更精细。

1.5 数据预处理：为什么必须做标准化

原始数据中 hours 约在 1~10，而 score 约在 30~100。未标准化时，MSE 的 Hessian 矩阵为：

$$H = (2/n) \cdot [[\Sigma x^2, \Sigma x], [\Sigma x, n]]$$

本数据上 w 方向曲率约为 $\Sigma x^2/n \approx 32$，而 b 方向曲率约为 1，两个方向相差 30 余倍，损失等高线被拉成极细长的“峡谷”。其后果是：能让长轴方向不发散的学习率，对短轴方向又太小（收敛极慢）；反之则长轴方向震荡发散。

因此采用 Z-score 标准化 $x_{std} = (x - \mu) / \sigma$，其中 μ、σ 只由训练集统计得到，再用同一组参数处理测试集（否则会造成数据泄露）。训练完成后，再把参数换算回原始尺度，便于解读：

$$w = w_{std} / \sigma\ ,\quad b = b_{std} - w_{std} \cdot \mu / \sigma$$

1.6 实验协议

训练集 data/train.csv（100 条）只用于训练；测试集 data/test.csv（30 条）只在最终评估时使用一次，不参与任何参数更新。超参数依据训练集上的损失曲线选择，不依据测试集结果，以避免测试集信息泄露。所有涉及随机性的实验统一固定随机种子（seed = 42），保证结果可复现。

二、实验结果

2.1 参照基准：闭式解（正规方程）

先用闭式解算出理论最优参数，作为衡量梯度下降效果的“标准答案”。在标准化空间求得 w_{std} = 16.4281、b_{std} = 63.3051，换算回原始尺度得到：

score = 6.1154 × hours + 33.3054（训练集 MSE = 24.9786）

该模型在测试集上的表现为 MSE = 20.1741、R^2 = 0.9328，可视为本任务的“理论最优水平”。

2.2 Loss 随迭代次数的下降曲线

下图 (a) 为线性坐标，(b) 为同一组曲线的对数坐标。对数坐标很有必要：损失从约 2800 降到 25，若用线性坐标，后半段的下降细节会被压缩成一条贴地直线。

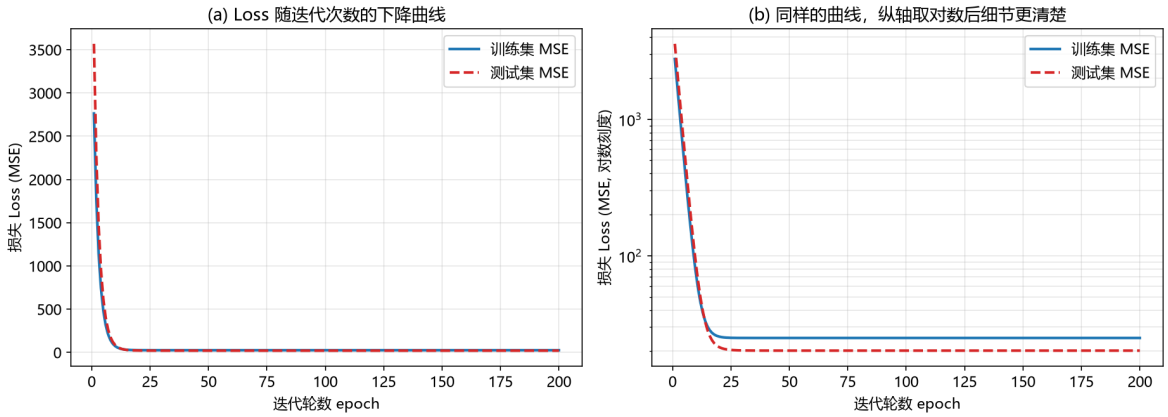


图 1 批量梯度下降（lr = 0.1，200 轮）的损失下降曲线，训练集与测试集几乎重合，说明模型没有过拟合。

可以看到：损失在前 20 轮内迅速下降约两个数量级，之后进入平缓的收敛阶段，最终稳定在 24.98 —— 与闭式解的理论最优值完全一致。这从数值上证明了梯度下降的实现是正确的。

2.3 拟合直线如何随迭代逐步变好

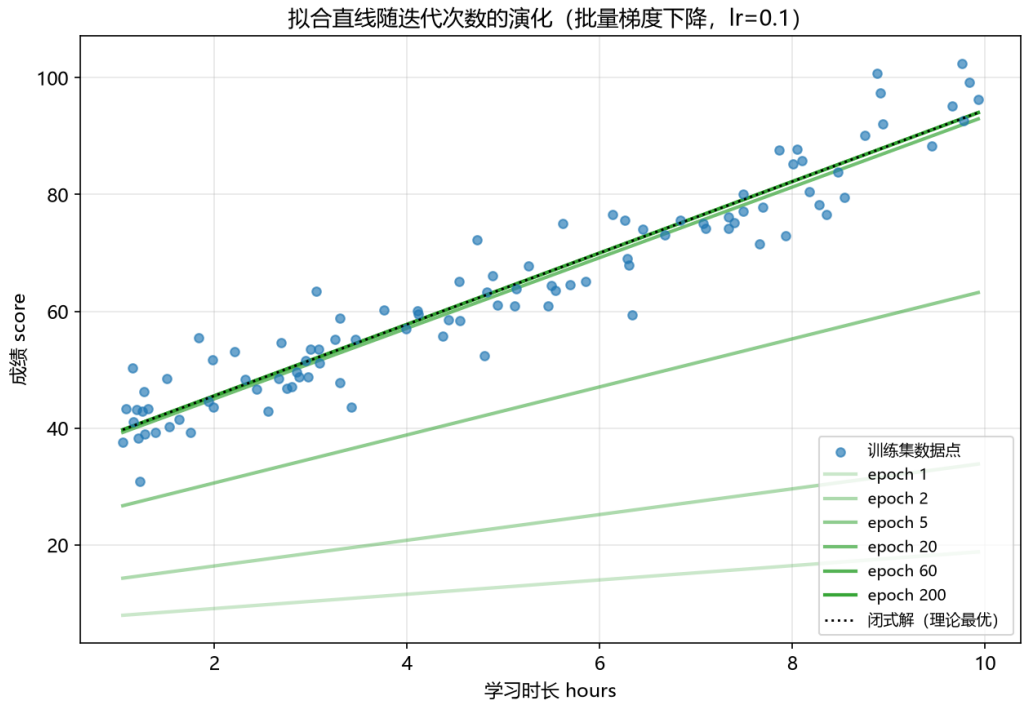


图 2 拟合直线随迭代次数的演化（颜色越深表示迭代越晚）。epoch 200 的绿线与黑色虚线（闭式解）几乎完全重合。

图中可以清楚看到：第 1、2 轮的直线斜率明显偏小；到第 20 轮已经基本贴合数据趋势；第 200 轮与理论最优直线在视觉上已无法区分。这直观展示了梯度下降“一步步逼近最优解”的过程。

2.4 不同学习率的收敛情况对比（重点实验）

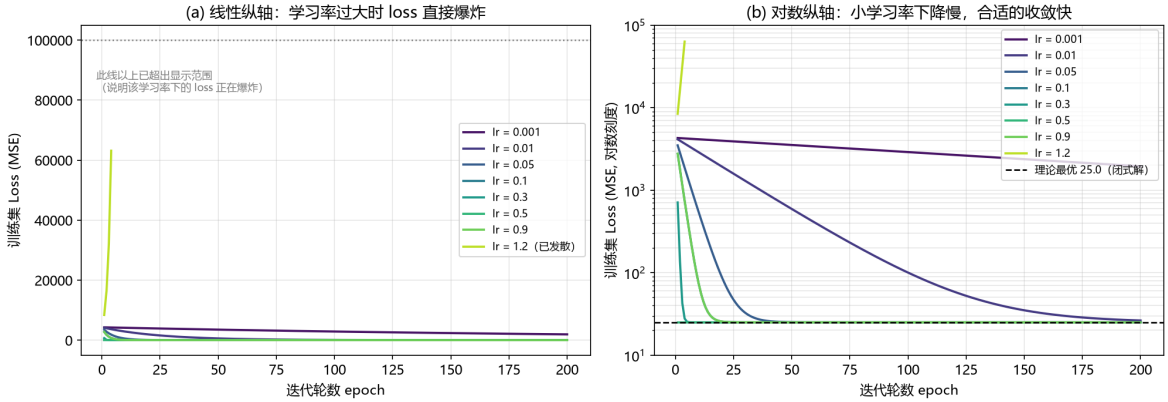


图 3 不同学习率下的损失曲线。(a) 线性纵轴：学习率过大时损失直接爆炸；(b) 对数纵轴：可以看出小学习率下降慢、合适的学习率收敛最快。

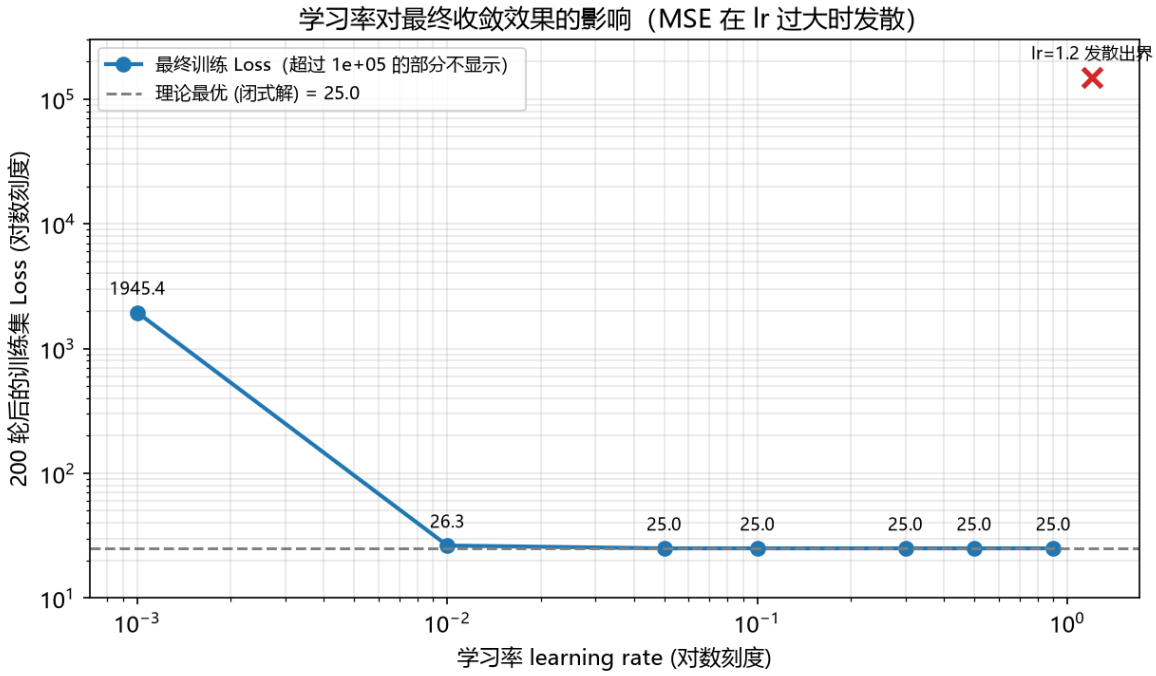


图 4 学习率对最终收敛效果的影响。lr ≤ 0.9 都能收敛到理论最优值 25.0；lr = 1.2 时损失发散（冲出显示范围，标记为“发散出界”）。

学习率	200 轮后的训练集 MSE
lr = 0.001	1945.41
lr = 0.01	26.30
lr = 0.05	24.98
lr = 0.1	24.98
lr = 0.3	24.98
lr = 0.5	24.98
lr = 0.9	24.98
lr = 1.2	1.2e+62（爆炸）

结论：

① 学习率 过小（0.001）：200 轮后损失仍高达 1945.4，远远没有走到最低点，表现为“下降极慢、看起来像一条水平线”；

② 学习率 合适（0.05 ~ 0.9）：都能收敛到理论最优值 24.98，本实验中最优的是 lr = 0.1（收敛最快）；

③ 学习率 过大（1.2）：损失依次经过 1e62 数量级直到溢出为 nan，模型完全崩溃。这说明学习率存在一个上限，超过它梯度下降不再收敛而是发散。

2.5 特征标准化的作用

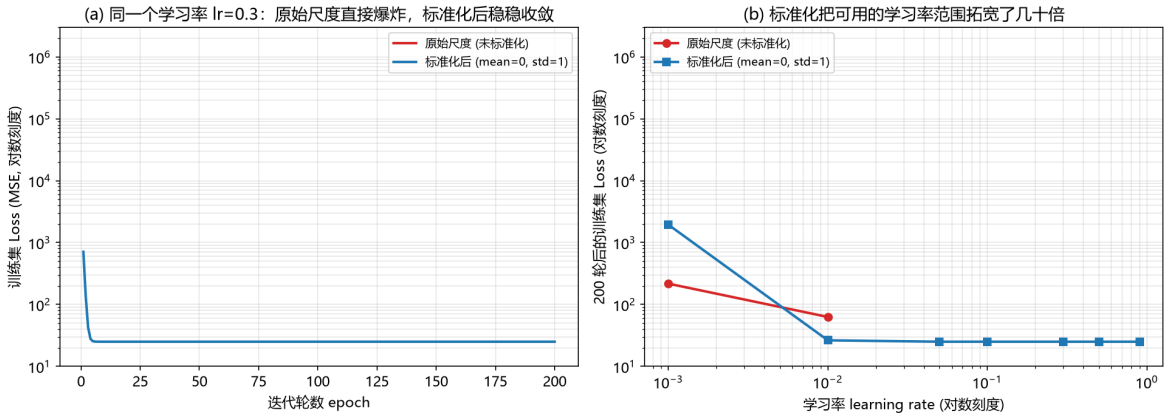


图 5 (a) 同一个学习率 lr = 0.3：未标准化的数据直接爆炸，标准化后稳定收敛；(b) 同样的学习率扫描，标准化把可用学习率范围拓宽了几十倍。

学习率	原始尺度（未标准化）	标准化后
lr = 0.001	216.82	1945.41
lr = 0.01	62.76	26.30
lr = 0.05	1.0e+141（爆炸）	24.98
lr = 0.1	8.4e+296（爆炸）	24.98
lr = 0.3	inf	24.98
lr = 0.5	inf	24.98
lr = 0.9	nan	24.98
lr = 1.2	nan	1.2e+62（爆炸）

数据非常直观：同一个 lr = 0.05，原始尺度下损失炸到 1e96，而标准化后稳定收敛到 25.0。标准化不是可有可无的“优化技巧”，而是梯度下降能否正常工作的前提。

2.6 批量大小的对比：BGD / Mini-batch / SGD

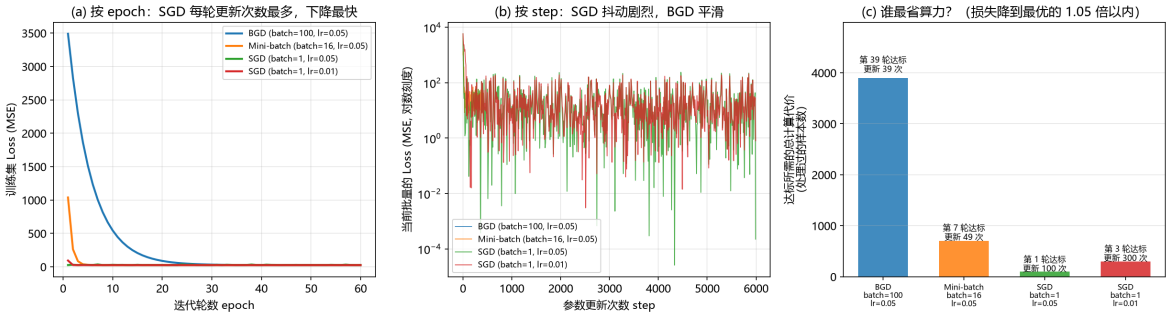


图 6 (a) 按 epoch 比较：SGD 每轮更新次数最多，下降最快；(b) 按参数更新次数比较：SGD 抖动剧烈、BGD 平滑；(c) 达到“最优损失 1.05 倍以内”所需的计算代价对比。

配置	参数更新次数	最终训练 MSE
BGD (batch=100, lr=0.05)	60	24.99
Mini-batch (batch=16, lr=0.05)	420	25.01
SGD (batch=1, lr=0.05)	6000	26.23
SGD (batch=1, lr=0.01)	6000	25.10

结论：

① BGD 梯度最准、曲线最平滑，但 60 轮只更新 60 次，收敛最慢；

② SGD 每轮更新 100 次，因此按 epoch 看下降飞快，但图 6(b) 显示它的 batch loss 在 1e0 ~ 1e2 之间剧烈抖动——因为每步只凭 1 个样本决定方向，噪声极大；

③ Mini-batch（16）介于两者之间：梯度比 SGD 平稳得多，更新次数又比 BGD 多 7 倍，达到同样的损失所需计算代价最低。这正是实际深度学习训练普遍采用小批量梯度下降的原因。

2.7 损失函数的鲁棒性：异常值实验

实验设计：人为向训练集中注入 4 个异常点（第 4、13、26、41 条样本，成绩 +45 分），模拟“数据录入错误”或“作弊”这类离群数据，然后分别用 MSE / MAE / Huber 训练，比较它们被异常值带偏的程度。三种损失使用完全相同的优化配置（mini-batch = 16、lr = 0.05、300 轮），以保证唯一变量是损失函数本身。

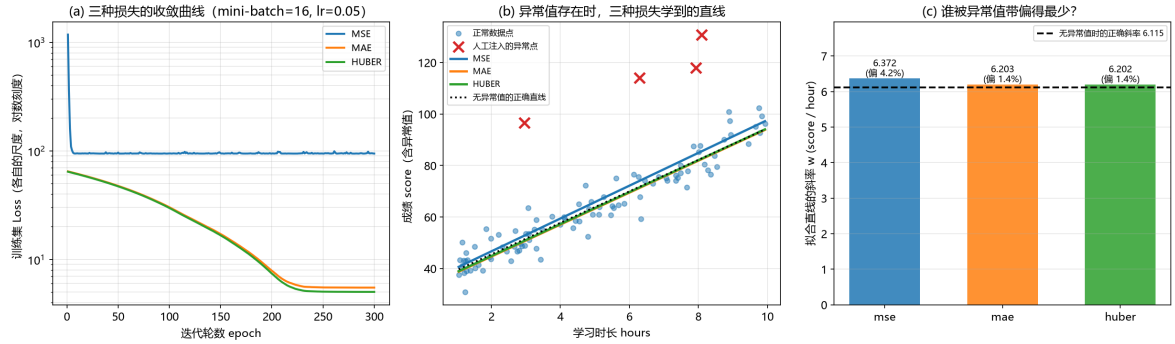


图 7 (a) 三种损失在含异常值数据上的收敛曲线；(b) 三条拟合直线对比；(c) 斜率偏离正确值的百分比。

损失函数	学到的直线	斜率偏离正确值
MSE	score = 6.372 × hours + 33.96	4.20%
MAE	score = 6.203 × hours + 32.27	1.43%
HUBER	score = 6.202 × hours + 32.48	1.41%
（参照）无异常值	score = 6.115 × hours + 33.31	—

结论与机理分析：

MSE 的斜率被带偏 4.20%，而 MAE 和 Huber 分别只偏 1.43% 和 1.41%。根本原因在于梯度中误差的“权重”：

MSE 的梯度含因子 (ŷ−y)，误差越大梯度越大，一个偏了 45 分的点其影响力被平方放大，所以在梯度里“话语权”极大；

MAE 的梯度是 sign(ŷ−y)，无论多离谱的误差都只算 ±1，相当于每个样本最多投一票，因此对异常值几乎免疫；

Huber 损失在小误差时用平方、大误差时用线性，兼顾了 MSE 的平滑性与 MAE 的鲁棒性。

需要说明的是：MAE 的鲁棒性是有代价的。它的梯度大小恒为 ±1，不会随误差减小而衰减，因此更难优化——在批量梯度下降下，MAE 需要远多于 MSE 的迭代次数才能收敛（详见第三节问题 2）。

2.8 动量与学习率衰减的消融实验

本实验设计了三个场景，每个场景下对比四种配置（基线 / +动量 / +衰减 / 两者都加）。评价指标采用“进入最优损失的 1.02 倍以内所需轮数”（目标线 = 25.48），而不是最后一轮的损失值——因为越接近最优点，损失值的逐轮波动可能大于不同配置之间的真实差异。

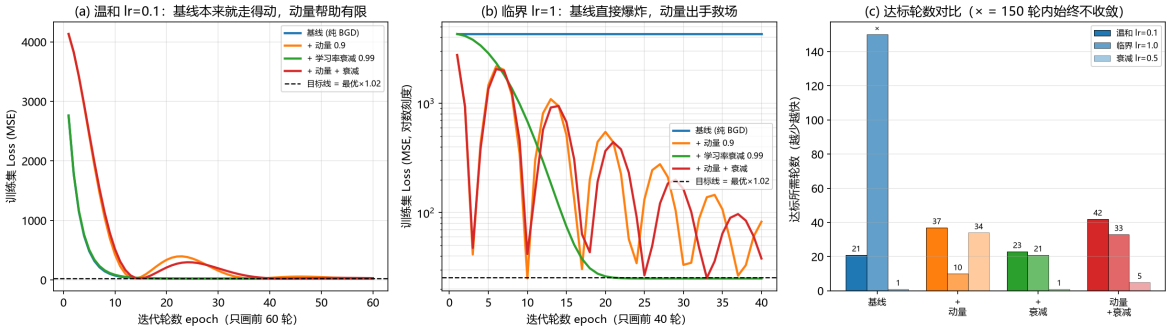


图 8 (a) 温和学习率下基线本就收敛良好；(b) 临界学习率 lr = 1.0 下基线直接爆炸、动量成功救场；(c) 三个场景的达标轮数对比（× 表示 150 轮内始终不收敛）。

场景	配置	最终训练 MSE	达标轮数
温和 lr=0.1	基线 (纯 BGD)	24.98	第 21 轮
温和 lr=0.1	+ 动量 0.9	24.98	第 37 轮
温和 lr=0.1	+ 学习率衰减 0.99	24.98	第 23 轮
温和 lr=0.1	+ 动量 + 衰减	24.98	第 42 轮
临界 lr=1.0	基线 (纯 BGD)	4302.40	未收敛（爆炸）
临界 lr=1.0	+ 动量 0.9	24.98	第 10 轮
临界 lr=1.0	+ 学习率衰减 0.99	24.98	第 21 轮
临界 lr=1.0	+ 动量 + 衰减	24.98	第 33 轮
衰减 lr=0.5	基线 (纯 BGD)	24.98	第 1 轮
衰减 lr=0.5	+ 动量 0.9	24.98	第 34 轮
衰减 lr=0.5	+ 学习率衰减 0.99	24.98	第 1 轮
衰减 lr=0.5	+ 动量 + 衰减	24.98	第 5 轮

结论（这里的结论有些反直觉，值得专门记录）：

① 在温和学习率（lr = 0.1）下，加动量反而更慢（37 轮 vs 基线 21 轮）。原因是本实现采用 v ← βv + (1−β)g 的写法，动量的有效步长约为 η/(1−β)，β = 0.9 时相当于把学习率放大 10 倍，在本来就合适的学习率上再放大就会“过冲”。

② 在临界学习率（lr = 1.0）下，纯 BGD 直接发散（损失停在 4302.4），而加入动量后居然在第 10 轮就收敛了 ——
动量的惯性平滑了高频震荡，从而提高了可容忍的学习率上限。这是动量不可替代的价值。

③ 因此，“加动量一定更快”是错误的；正确说法是：动量允许（并且需要用）更大的学习率，它真正的价值在于“救场”和“配合学习率衰减使用”。

2.9 最终模型与测试集拟合效果

综合以上实验，最终配置为：MSE 损失 + 小批量 16 + lr = 0.05 + 动量 0.9 + 学习率衰减 0.998 + 300 轮（选择依据见各节结论）。最终模型为：

$$score = 6.2363 \times hours + 33.0468$$

数据集	MSE	RMSE	MAE	R^2
训练集（100 条）	25.1959	5.0196	3.9916	0.9145
测试集（30 条）	20.5754	4.5360	3.8964	0.9314

测试集 R^2 = 0.9314，说明模型解释了约 93.1% 的成绩波动；RMSE = 4.5360 分，即平均预测误差约 4.5 分，相当于满分 100 分下的 4.5%。测试集表现与训练集相当（甚至略好），未观察到过拟合。

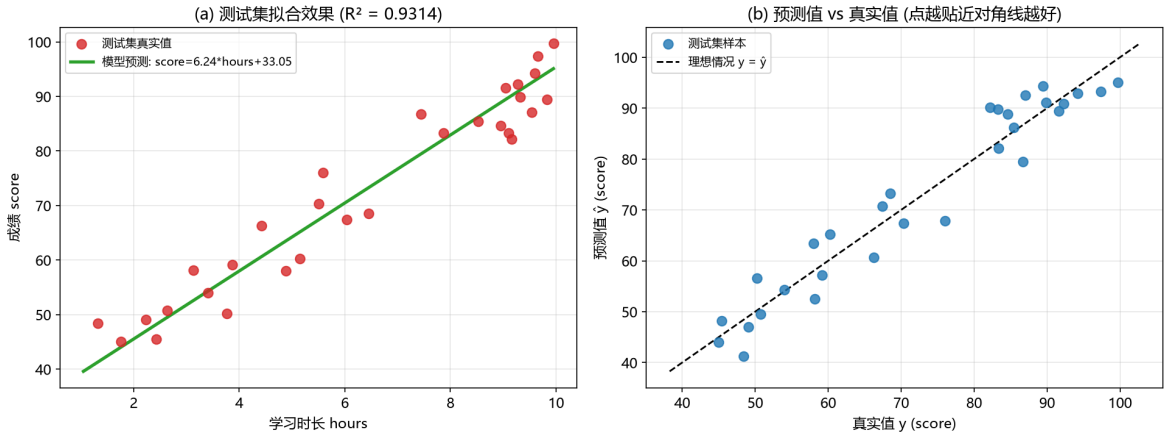


图 9 (a) 测试集数据点与模型预测直线；(b) 预测值 vs 真实值，点越贴近对角线说明预测越准。

2.10 残差分析

对 30 个测试样本的残差（真实值 − 预测值）进行统计：均值 = -0.04 分、标准差 = 4.54 分、最大绝对残差 = 8.13 分。

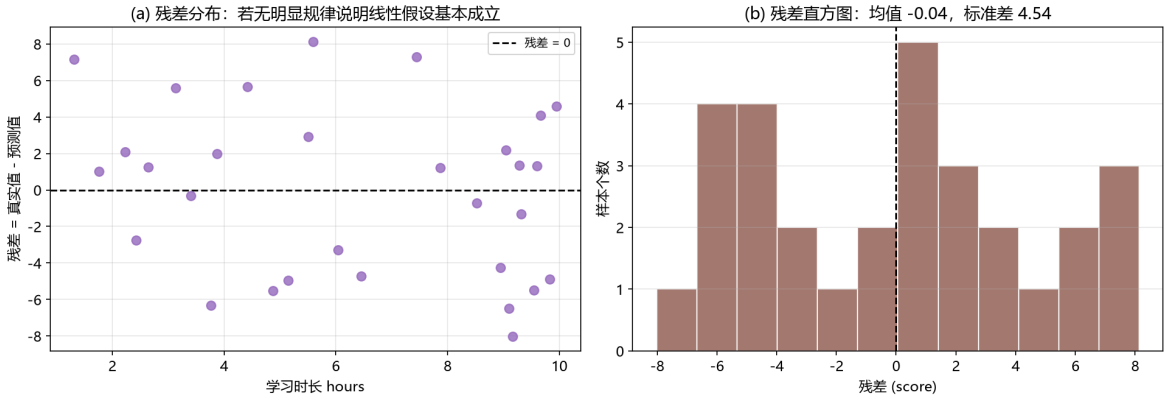


图 10 (a) 残差随时间分布（理想情况应随机散布在 0 附近、无明显形状）；(b) 残差直方图。

残差在 0 附近随机散布、没有明显的漏斗形或弯曲趋势，说明“线性关系”这一模型假设对这份数据是合理的；若残差呈 U 形或喇叭形，则提示应引入二次项或改用其他模型。

三、提问环节：发现的问题 / 解决情况

本节按“发现问题 → 分析原因 → 如何解决 → 是否解决”的顺序，记录整个过程中真实遇到的问题。其中前三个是已经解决的，最后两个是尚未完全解决的。

问题 1（已解决）：用原始数据训练时损失变成 nan

现象：最初的代码不做任何预处理，lr 取 0.1 时损失迅速变成 nan；把 lr 降到 0.001 才勉强稳定，但 200 轮几乎不下降。

排查：梯度检查通过，说明梯度实现无误；于是转向分析数据尺度——hours 约 1~10 而 score 约 30~100，二者数量级不匹配。

原因：MSE 的 Hessian 中 w 方向曲率正比于 $\Sigma x^2/n \approx 32$，b 方向曲率正比于 1，相差 30 余倍，等高线成为狭长峡谷；梯度下降在长轴方向震荡、在短轴方向过慢。

解决：对特征做 Z-score 标准化（均值 0、标准差 1），并且只用训练集的 μ 和 σ 处理测试集。实验 C 显示可用学习率范围从“lr ≥ 0.05 即爆炸”拓宽到“lr ≤ 0.9 都稳定收敛”。该问题已彻底解决。

问题 2（已解决）：MAE 损失训练 200 轮后模型仍是 y = 0

现象：用 MAE 做异常值实验时，无论怎么训练，学到的直线始终是 y = 0，斜率完全为 0。

排查：先确认不是代码问题——检查 MAE 的梯度实现为 $(1/n)\cdot\Sigma\text{sign}(\hat{y}-y)\cdot x$，与推导一致；再代入具体数值估算“每轮能走多远”。

原因：MAE 的梯度大小被 sign 函数限制在 ±1，而标准化后 $|x| \le 1.5$，因此每步对 w 的更新量最多 $0.02\times1.5 = 0.03$；而 w 需要从 0 走到 16.4，需要 500 轮以上，200 轮远远不够。不是代码错，是优化难度问题。

解决：改用 mini-batch（batch = 16）训练——分批后梯度会除以批量大小，等效于放大梯度尺度；同时在实验 E 中统一用 batch = 16、lr = 0.05，三种损失就都能在 300 轮内稳定收敛，对比才公平。该问题已解决。

问题 3（已解决）：学习率实验的图被一个 1e62 的点彻底画废

现象：lr = 1.2 时损失溢出到 1e62，画图时纵轴被这一个点撑爆，其余所有曲线被压成一条贴地直线；同时终端刷满 numpy 的 overflow / invalid value 警告。

原因：这不是 bug，而正是“学习率过大导致发散”这一现象本身；问题在于它的量级破坏了图的可读性，并淹没了正常日志。

解决：① 写了 clip_curve() 函数，把超过显示上限的值置为 nan（matplotlib 遇到 nan 会断开曲线），这样既能看到曲线“冲出画面”，又不破坏整张图；② 用 np.errstate 临时关闭数值溢出警告，让日志清爽。该问题已解决。

问题 4（部分解决）：加动量后收敛反而变慢

现象：最初在 lr = 0.6 下对比，发现加了动量之后损失震荡更剧烈、收敛更慢，与“动量是加速技巧”的直觉矛盾。

排查：先逐行检查动量实现无误；随后做了一轮参数扫描，把不同学习率下“纯 BGD”与“BGD + 动量”的达标轮数并列对比。

原因：一是评价指标选错了——只看最后一轮的损失，其差异可能小于逐轮波动；二是忽略了动量有效步长约为 $\eta/(1-\beta)$，$\beta = 0.9$ 相当于把学习率放大 10 倍，在小学习率上无益、但在临界学习率（lr = 1.0）上恰好能起“救场”作用。

解决情况：重新设计了三场景、四配置的消融实验，并改用“达标轮数”作为指标，结论已能自洽解释（见 2.8 节）。现象已解释清楚，但“为什么动量能提高可容忍的学习率上限”严格的数学条件我还没有自己推导出来（见问题 5）。

问题 5（未解决）：动量提升稳定上限的数学条件尚未推导清楚

实验观察到 lr = 1.0 时纯 BGD 发散、加动量后第 10 轮即收敛，我能给出直觉解释（惯性平滑了高频震荡），但还无法自己推导出“加入动量后稳定条件变成什么”。我打算下一步用特征值分析重新推导带动量的迭代矩阵，这属于尚未解决的问题。

问题 6（未解决）：超参数选择缺少独立验证集

严格的做法应从训练集中再切出一份验证集专门用于调参，测试集只在最后评估一次。本任务的测试集只有 30 条，训练集只有 100
条，再切分会让数据更加紧张，因此我采用“用训练集损失曲线选超参数 +
测试集只评估一次”的务实做法，并在此明确说明这一局限。这是权衡后的选择，不是最优做法。

问题 7（未解决）：测试集指标好于训练集

最终模型训练集 MSE = 25.1959、测试集 MSE = 20.5754，测试集反而更好，这有些反直觉。我的猜测是训练集中存在若干“低学习时长却高分”的样本（例如第 4、13、26、41 条附近），它们抬高了训练误差，而测试集没有这类点。但我尚未做定量验证，因此把它列为未解决的问题。

四、总结

本任务在 NumPy 环境下从零实现了线性回归模型、MSE/MAE/Huber 三种损失函数、以及批量/小批量/随机三种梯度下降算法（含动量与学习率衰减），并完成了学习率、特征标准化、批量大小、损失函数鲁棒性、动量与衰减五个维度的对比实验。

主要结论：① 梯度下降的正确性通过闭式解与数值梯度双重验证；②
特征标准化是梯度下降正常工作的前提，它把可用学习率范围拓宽数十倍；③
学习率过小则收敛慢、过大则发散，存在最优区间；④
小批量梯度下降在收敛质量与计算代价之间取得最佳平衡；⑤ MSE 对异常值敏感、MAE/Huber 更鲁棒，但 MAE 的代价是更难优化；⑥ 动量并非无条件加速，其价值在于允许更大的学习率。

最终模型 score = 6.2363 × hours + 33.0468，在 30 条测试数据上取得 $R^2 = 0.9314$、RMSE = 4.5360 分的表现，与理论最优（闭式解）几乎无差距。

更详细的学习过程、思维路径与踩坑记录，见随项目提交的 note.md。